

Análise empírica de algoritmos de ordenação: Bubble Sort aprimorado e Radix Sort

com comparação a uma baseline de quatro algoritmos

Eric Hiyosuke Nakau Azuma¹ · Pedro Luiz Yugata Duarte da Silva Sinzato¹

¹ Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, SP — NUSP 17887791 e 17919490

Resumo. Este trabalho apresenta uma análise empírica de desempenho de algoritmos de ordenação. Foram implementados e avaliados o Bubble Sort aprimorado — com interrupção antecipada quando o vetor já está ordenado — e o Radix Sort, em entradas de 100 a 10.000 elementos e em quatro configurações: ordenadas, ordenadas em ordem decrescente, aleatorizadas e aleatorizadas com repetição. Os tempos de ordenação, medidos com o QueryPerformanceCounter, correspondem à média de nove execuções. O Radix Sort foi mais rápido em todos os casos, exceto na entrada já ordenada, em que o Bubble Sort aprimorado foi de 15 a 29 vezes mais rápido; para 10.000 elementos em ordem decrescente, o Radix Sort foi cerca de 376 vezes mais rápido. Em uma segunda parte, o comportamento de quatro algoritmos fornecidos como baseline (A a D) foi aproximado por curvas $O(n^2)$ (A), $O(n)$ (B e C) e $O(n \log n)$ (D). O algoritmo C, linear porém o mais lento na maioria dos testes, ilustra que a análise assintótica não descreve o desempenho por completo.

Palavras-chave: análise de algoritmos; ordenação; Bubble Sort; Radix Sort; complexidade assintótica.

Abstract. This paper presents an empirical performance analysis of sorting algorithms. An enhanced Bubble Sort (with early termination on sorted input) and Radix Sort were implemented and timed on inputs of 100 to 10,000 elements under four configurations: sorted, reverse-sorted, random, and random with repeated values. Each reported time is the mean of nine runs measured with the Windows QueryPerformanceCounter. Radix Sort was faster in every case except sorted input, where the enhanced Bubble Sort was 15 to 29 times faster; for 10,000 reverse-sorted elements, Radix Sort was about 376 times faster. In a second part, four baseline algorithms (A–D) were fitted to asymptotic curves, behaving approximately as $O(n^2)$ (A), $O(n)$ (B and C) and $O(n \log n)$ (D). Algorithm C, linear yet the slowest in most tests, shows that asymptotic analysis alone does not fully describe practical performance.

Keywords: algorithm analysis; sorting; Bubble Sort; Radix Sort; asymptotic complexity.

1 INTRODUÇÃO

A análise de algoritmos busca descrever como o tempo de execução e o uso de memória de um programa crescem com o tamanho da entrada. A notação assintótica (Big O) resume esse crescimento, mas ignora constantes multiplicativas, custos de alocação e características da máquina; por isso, medições empíricas são um complemento natural à análise teórica.

Este trabalho, desenvolvido no Projeto 1 da disciplina ICC2, tem duas partes. Na primeira, dois algoritmos de ordenação — o Bubble Sort aprimorado e o Radix Sort — foram implementados em linguagem C e cronometrados sobre quatro tipos de entrada. Na segunda, analisa-se uma baseline de quatro algoritmos fornecidos no enunciado (A, B, C e D), cujo comportamento assintótico foi inferido a partir dos tempos informados.

Os dados da baseline foram coletados, ao que se pôde apurar, com a biblioteca `time.h` no computador de quem elaborou o enunciado, enquanto os tempos dos algoritmos implementados foram medidos com o QueryPerformanceCounter da biblioteca `windows.h`. Como as máquinas e os instrumentos de medição diferem, o ruído entre os dois conjuntos é considerável; por isso, as

análises foram mantidas separadas e não se comparam diretamente tempos da baseline com os dos algoritmos implementados.

2 METODOLOGIA

Os algoritmos implementados foram executados sobre as mesmas entradas fornecidas para a baseline, com $N \in \{100; 500; 1.000; 5.000; 10.000\}$ elementos e quatro tipos de entrada: ordenados, ordenados em ordem decrescente (decrescentes), aleatorizados e aleatorizados com repetição de valores. Cada caso foi executado em dez rodadas; a primeira foi descartada, e os valores reportados são a média aritmética das nove rodadas restantes.

Foi medido apenas o tempo de ordenação, excluindo-se os tempos de leitura da entrada e de escrita da resposta, por meio das funções QueryPerformanceCounter e QueryPerformanceFrequency da biblioteca `windows.h`. Os tempos dos algoritmos implementados são apresentados em microssegundos (μs); os da baseline, em milissegundos (ms).

Duas medidas de dispersão aparecem neste trabalho. No Apêndice B, o desvio padrão de cada caso é a média dos desvios padrão informados nas nove rodadas. Nas Tabelas 3 e 4, o desvio padrão (σ) é calculado entre os quatro tipos

de entrada (desvio padrão populacional) e indica, portanto, o quanto o tempo do algoritmo varia conforme a natureza dos dados.

Tabela 1 – Complexidade assintótica de tempo dos algoritmos implementados.

Algoritmo	Melhor	Médio	Pior
Bubble Sort aprimorado	$O(n)$	$O(n^2)$	$O(n^2)$
Radix Sort	$O(n \cdot D)$	$O(n \cdot D)$	$O(n \cdot D)$

Fonte: elaboração própria. D é o número de dígitos do maior valor do vetor.

3 ALGORITMOS IMPLEMENTADOS

3.1 Bubble Sort aprimorado

O Bubble Sort percorre o vetor repetidamente, comparando elementos vizinhos e trocando-os de posição quando estão fora de ordem; a cada passagem, o maior elemento ainda não posicionado é deslocado para o fim do vetor (código no Apêndice A, conforme Sedgewick). A versão aprimorada usa a variável de controle troca: se uma passagem completa não realiza nenhuma troca, o vetor já está ordenado e o laço principal é interrompido.

O pior caso ocorre com entradas em ordem decrescente: todas as passagens são necessárias e o algoritmo realiza $O(n^2)$ comparações e trocas. O caso médio também é $O(n^2)$. No melhor caso, com o vetor já ordenado, uma única passagem de comparações basta, e a complexidade é $O(n)$.

3.2 Radix Sort

O Radix Sort começa localizando o maior valor do vetor e, em seguida, ordena os elementos dígito a dígito, do menos para o mais significativo, aplicando a cada etapa uma ordenação por contagem (Apêndice A, adaptado de GeeksforGeeks). A Tabela 2 ilustra o processo; na terceira passagem, números com menos dígitos são tratados com zeros à esquerda.

Tabela 2 – Passagens do Radix Sort sobre um vetor de exemplo.

Etapas	Vetor
Entrada	25, 57, 148, 37, 12, 91, 86, 33
1ª (unidades)	91, 12, 33, 25, 86, 57, 37, 148
2ª (dezenas)	12, 25, 33, 37, 148, 57, 86, 91
3ª (centenas)	012, 025, 033, 037, 057, 086, 091, 148

Fonte: elaboração própria.

O vetor é percorrido uma vez para obter o maior valor e uma vez para cada dígito desse valor, o que resulta em $O(n \cdot D)$ operações, com $D = 1 + \lfloor \log_{10} M \rfloor$ e M o maior valor do vetor. Como D é limitado para inteiros de tamanho fixo, o algoritmo tende a se comportar como $O(n)$. Em contrapartida, por não ser baseado em comparações, o Radix Sort exige memória auxiliar (vetor de saída e contadores), e o tempo gasto para alocá-la e utilizá-la eleva a constante do algoritmo.

4 RESULTADOS E DISCUSSÃO

As Tabelas 3 e 4 apresentam os tempos médios de ordenação e a Figura 1 os mostra em escala logarítmica. O Apêndice B traz os valores completos, com os desvios padrão de cada caso.

Tabela 3 – Bubble Sort aprimorado: tempo médio de ordenação (μs) e desvio padrão (σ) entre os tipos de entrada.

N	Ordenados	Decresc.	Aleator.	Aleat. c/ rep.	σ
100	0,346	31,53	30,01	24,42	12,54
500	1,054	731,92	530,87	576,77	275,33
1.000	2,358	2.707,55	2.197,28	2.115,37	1.037,37
5.000	13,19	57.661,07	44.905,02	44.792,16	21.897,56
10.000	21,02	208.667,49	188.697,32	197.805,50	86.186,77

Fonte: elaboração própria.

Tabela 4 – Radix Sort: tempo médio de ordenação (μs) e desvio padrão (σ) entre os tipos de entrada.

N	Ordenados	Decresc.	Aleator.	Aleat. c/ rep.	σ
100	5,322	4,064	4,196	4,086	0,525
500	17,14	16,48	15,87	18,74	1,071
1.000	52,19	41,70	41,56	41,85	4,545
5.000	215,95	186,64	260,77	188,73	29,89
10.000	609,36	554,34	503,79	428,43	66,61

Fonte: elaboração própria.

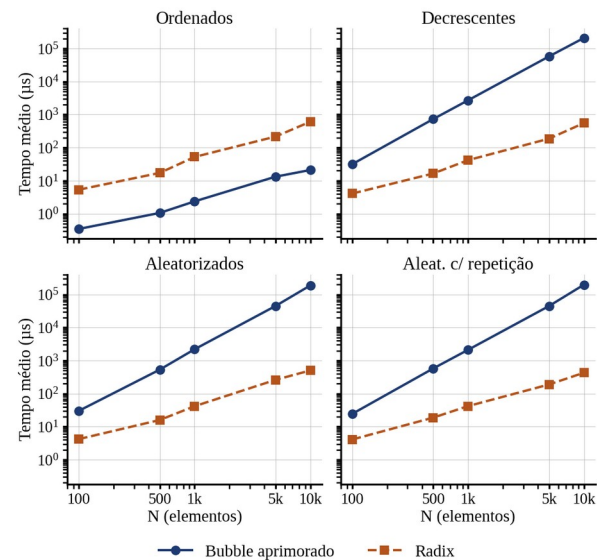


Figura 1 – Tempo médio de ordenação (μs) em função de N , por tipo de entrada (escala log-log).

Fonte: elaboração própria.

Entradas ordenadas. O Bubble Sort aprimorado foi muito mais rápido que o Radix Sort em todos os tamanhos (Tabela 5): para $N = 10.000$, 21,02 μs contra 609,36 μs . O resultado é compatível com a análise assintótica — ambos são lineares nesse caso, $O(n)$ e $O(n \cdot D)$ —, mas evidencia o peso da constante: o Bubble Sort faz uma única passagem de comparações, ao passo que o Radix Sort

executa todas as passagens por dígito e copia o vetor a cada uma delas, qualquer que seja a ordem inicial dos dados.

Tabela 5 – Entradas ordenadas: tempo médio (μ s) de cada algoritmo.

N	Bubble	Radix	Razão	Acréscimo
100	0,346	5,322	15,40×	1.440,09%
500	1,054	17,14	16,25×	1.525,31%
1.000	2,358	52,19	22,14×	2.113,71%
5.000	13,19	215,95	16,38×	1.537,50%
10.000	21,02	609,36	28,99×	2.798,79%

Fonte: elaboração própria. Razão = Radix/Bubble; acréscimo = Radix/Bubble - 1.

Entradas decrescentes. O Radix Sort foi muito superior, e a diferença cresce com N: para N = 10.000, ordenou em 554,34 μ s, contra 208.667,49 μ s do Bubble Sort aprimorado (cerca de 376 vezes mais rápido); já em N = 100 a vantagem é de aproximadamente 7,8 vezes. Ao contrário do que ocorreu com as entradas ordenadas, esse tipo de entrada correspondeu ao esperado: o crescimento quadrático do Bubble Sort é nítido na Figura 1, e esse é o seu pior caso, com o maior tempo entre os quatro tipos de entrada em todos os valores de N (Tabela 3).

Entradas aleatorizadas. Os resultados foram similares aos das entradas decrescentes. Vale ressaltar que o Bubble Sort foi um pouco mais rápido que naquele caso (188.697,32 μ s contra 208.667,49 μ s em N = 10.000), enquanto o Radix Sort manteve tempos da mesma ordem de grandeza (503,79 μ s).

Entradas aleatorizadas com repetição. Os dois algoritmos mantiveram tempos semelhantes aos do caso anterior. Para o Bubble Sort, a diferença relativa entre os dois tipos de entrada aleatória variou de -19% (N = 100) a +9% (N = 500) e ficou abaixo de 5% para N $\geq$ 1.000. O Radix Sort apresentou maior oscilação em alguns pontos, como em N = 5.000 aleatorizados (260,77 μ s, com desvio padrão de 176,99 μ s, contra 188,73 μ s com repetição).

Consistência. O Radix Sort apresentou comportamento muito mais consistente entre os tipos de entrada: o σ das Tabelas 3 e 4 varia de 0,53 a 66,61 μ s para o Radix Sort e de 12,54 a 86.186,77 μ s para o Bubble Sort aprimorado. A única exceção ao melhor desempenho do Radix Sort é a entrada já ordenada.

5 ANÁLISE DA BASELINE

A baseline é formada por quatro algoritmos (A a D), com tempos em ms fornecidos para os mesmos tamanhos e tipos de entrada. Para inferir o comportamento assintótico de cada um, os pontos foram inseridos no GeoGebra e o parâmetro c da definição $T(n) \leq c \cdot f(n)$ foi ajustado manualmente, por tentativa, até que a curva se assemelhasse ao formato dos pontos. Os algoritmos foram graficados separadamente (Figura 2) para evitar distorções de escala. Os tempos em N = 100 iguais a zero

provavelmente ficaram abaixo da resolução do relógio utilizado.

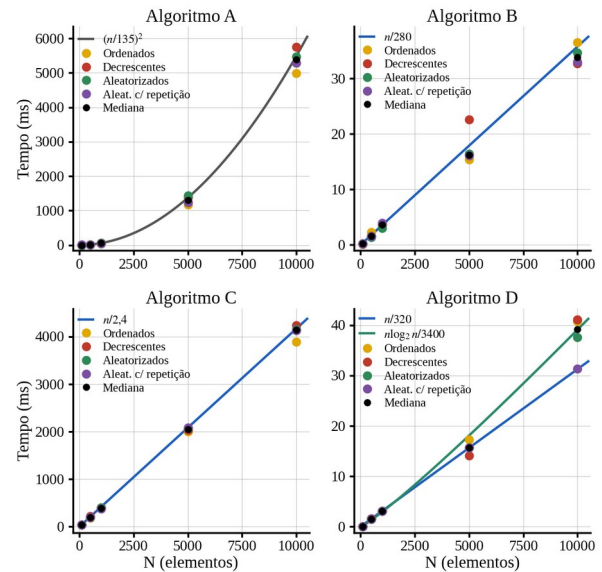


Figura 2 – Tempos da baseline (ms) e curvas ajustadas manualmente. Amarelo: ordenados; vermelho: decrescentes; verde: aleatorizados; roxo: aleatorizados com repetição; preto: mediana dos quatro tipos.

Fonte: elaboração própria, a partir dos dados da baseline.

Tabela 6 – Baseline: tempo mediano (ms) entre os quatro tipos de entrada.

Alg.	100	500	1.000	5.000	10.000
A $O(n^2)$	0,348	10,470	52,160	1.310,80	5.383,15
B $O(n)$	0,199	1,597	3,687	16,255	33,800
C $O(n)$	41,675	197,35	395,10	2.052,00	4.142,70
D $O(n \log n)$	0,000	1,563	3,128	15,718	39,185

Fonte: elaboração própria, a partir dos dados da baseline.

Tabela 7 – Baseline: desvio padrão (ms) entre os quatro tipos de entrada.

Alg.	100	500	1.000	5.000	10.000
A	0,237	1,645	9,154	104,94	275,49
B	0,044	0,341	0,396	2,877	1,504
C	2,648	11,576	10,599	32,639	130,07
D	0	0,007	0,015	1,122	3,906

Fonte: elaboração própria, a partir dos dados da baseline.

Algoritmo A. Comportamento semelhante a $O(n^2)$, consistente para todos os tipos de entrada (curva $f(n) = (n/135)^2$). O pior caso parece ser o das entradas decrescentes (vermelho), que costumam estar entre as mais lentas e são as mais lentas em N = 10.000 (5.752,4 ms). O melhor caso parece ser o das entradas ordenadas (amarelo), consistentemente mais rápidas à medida que N aumenta. Fora isso, é difícil inferir mais: as oscilações entre tipos são inconsistentes, e não é possível determinar se vêm do algoritmo ou do ruído da coleta empírica.

Algoritmo B. Comportamento semelhante a $O(n)$, com relativa consistência ($f(n) = n/280$). A principal discrepância é o teste com 5.000 entradas decrescentes (22,54 ms, contra 15,36 a 16,45 ms nos demais tipos).

Fora isso, as velocidades oscilam em torno do padrão linear de forma aparentemente aleatória e com variância pequena, o que permite atribuir as diferenças ao ruído de outros processos na máquina de quem forneceu a baseline, sem que se possam determinar melhores e piores casos.

Algoritmo C. Comportamento semelhante a $O(n)$, com relativa consistência ($f(n) = n/2,4$). O melhor caso parece ser o das entradas ordenadas: em $N = 100$ são ligeiramente mais lentas que as demais, mas nos tamanhos maiores tendem a ser mais rápidas. Fora isso, as oscilações são pequenas e podem ser atribuídas ao ruído da coleta.

Algoritmo D. Em geral, comportamento semelhante a $O(n \log n)$ (curva verde, $g(n) = n \cdot \log_2 n/3400$), com um melhor caso que parece $O(n)$ (reta azul, $f(n) = n/320$). Nos primeiros testes o algoritmo parece seguir $O(n)$, embora com variância considerável em $N = 5.000$; nos dois últimos, e especialmente em $N = 10.000$, os pontos se aproximam mais de $O(n \log n)$ em todos os tipos, exceto nas entradas aleatorizadas com repetição (roxo), que ainda se assemelham a $O(n)$: em $N = 10.000$ seu tempo (31,39 ms) fica próximo da reta $n/320$ (31,25 ms). Isso pode ser explicado pelo fato de a divergência entre $O(n)$ e $O(n \log n)$ ser pouco notável para valores pequenos de n . Nos maiores tamanhos ($N = 5.000$ e 10.000), as entradas ordenadas (amarelo) estão entre as mais lentas, enquanto as decrescentes (vermelho) têm desempenho menos previsível, oscilando entre o mais rápido e o mais lento — não é possível dizer se por peculiaridade da implementação ou por ruído da coleta.

Comparação. O algoritmo mais lento para entradas grandes é o A, tanto pelos dados quanto por sua análise assintótica $O(n^2)$. Para entradas menores, o C é consistentemente pior que o A até $N = 5.000$ (2.052,00 contra 1.310,80 ms); entre 5.000 e 10.000, o A passa a ser mais lento (5.383,15 contra 4.142,70 ms), por conta de seu crescimento quadrático. O B é o mais eficiente para entradas grandes, com tempos um pouco menores que os de D (33,80 contra 39,19 ms em $N = 10.000$) e comportamento que parece $O(n)$, mais eficiente que o $O(n \log n)$ de D, supondo que ambas as análises estejam corretas. Uma observação curiosa é o algoritmo C: embora pareça linear, apresentou tempos consistentemente maiores que os demais em quase todos os casos (41,68 ms em $N = 100$, contra 0,20 ms do B), o que lembra que a análise assintótica não diz tudo sobre um algoritmo.

Por tipo de entrada, as conclusões são semelhantes. Com entradas ordenadas e decrescentes, o B é o mais rápido, com o D logo atrás; C e A são os mais lentos (C para entradas menores e A para as maiores). Com entradas aleatorizadas, B e D ficam quase empatados, com o B um pouco à frente (talvez por ruído). Com entradas aleatorizadas com repetição, D e B também quase empatam, com o D um pouco à frente (talvez por ruído); C e A mantêm o comportamento dos demais casos.

6 CONCLUSÃO

Entre os algoritmos implementados, o Radix Sort mostrou-se muito superior para valores maiores de N e apresentou comportamento mais consistente em todo o espectro de entradas. A única exceção foi a entrada já ordenada, em que o Bubble Sort aprimorado foi de 15 a 29 vezes mais rápido (Tabela 5). O Bubble Sort aprimorado só se mostra competitivo para entradas pequenas ($N < 100$) — faixa não avaliada, pois já em $N = 100$ o Radix Sort foi cerca de 6 a 8 vezes mais rápido nas entradas desordenadas — e em entradas ordenadas, tendo as entradas decrescentes como pior caso.

Na baseline, o algoritmo A aproximou-se de $O(n^2)$, B e C de $O(n)$ e D de $O(n \log n)$, com melhor caso linear. O B foi o mais eficiente para entradas grandes, e o C — linear, porém com constante elevada — foi o mais lento na maioria dos testes. Esse resultado, somado ao desempenho do Bubble Sort em entradas ordenadas, reforça que a análise assintótica deve ser complementada por medições.

Como limitação, os resultados dependem da máquina e do instrumento de medição, e as curvas da baseline foram ajustadas manualmente, o que confere caráter qualitativo às classificações propostas.

REFERÊNCIAS

SEGEWICK, R. *Algorithms in C*. Addison-Wesley.

GEEKSFORGEES. *C Program for Radix Sort*. Disponível em: <https://www.geeksforgeeks.org/c-program-for-radix-sort/>. Acesso em: out. 2026.

APÊNDICE A – CÓDIGOS-FONTE

Código 1 – Bubble Sort aprimorado.

```
void bubblesort_aprimorado (int v[], int n)
{
    int j, aux, troca = 1;
    for (int i = 0; i < n && troca; i++)
    {
        troca = 0;
        for (j = 0; j < n - i - 1; j++)
        {
            if (v[j] > v[j+1])
            {
                troca = 1;
                aux = v[j];
                v[j] = v[j+1];
                v[j+1] = aux;
            }
        }
    }
    return;
}
```

Fonte: Sedgewick, *Algorithms in C*; adaptado pelos autores.

Código 2 – Radix Sort.

```
// Funcao auxiliar que pega o maior valor no arr[]
int getMax(int arr[], int n)
{
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

// Funcao que faz o ordenamento por contagem do arr[] a partir do algarismo indicado pelo int exp
void countSort(int arr[], int n, int exp)
{
    // output array
    int output[n];
    int i, count[10] = { 0 };

    // Armazena a quantidade de ocorrencias de cada algarismo em count[]
    for (i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Muda count[i] para que count[i] agora contenha a verdadeira posicao
    // desse digito em output[]
    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Constroi o output array
    for (i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copia o output array em arr[], fazendo com que arr[] agora contenha
    // os numeros organizados corretamente de acordo com o digito atual
    for (i = 0; i < n; i++)
        arr[i] = output[i];
}

// Funcao principal que organiza o arr[] de tamanho n usando Radix sort
void radixsort(int arr[], int n)
{
    // encontra o maior numero para saber o numero de digitos
    int m = getMax(arr, n);

    // Roda o countSort (ordenacao por contagem) para cada digito.
    // Note que o digito exp, equivalente a 10^i, e passado
    // com i sendo o digito atual sendo analisado
    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}
```

Fonte: GeeksforGeeks, *C Program for Radix Sort*; adaptado pelos autores.

APÊNDICE B – TEMPOS E DESVIOS PADRÃO DE CADA CASO**Tabela B1** – Tempo médio e desvio padrão (μ s) de cada algoritmo, por tamanho e tipo de entrada.

N	Tipo de entrada	Bubble: média	Bubble: desvio	Radix: média	Radix: desvio
100	Ordenados	0,346	0,074	5,322	1,176
100	Decrescentes	31,53	6,104	4,064	0,435
100	Aleatorizados	30,01	11,50	4,196	0,390
100	Aleat. c/ repetição	24,42	4,533	4,086	0,395
500	Ordenados	1,054	0,131	17,14	0,769
500	Decrescentes	731,92	131,43	16,48	0,946
500	Aleatorizados	530,87	59,34	15,87	0,579
500	Aleat. c/ repetição	576,77	104,01	18,74	1,294
1.000	Ordenados	2,358	0,236	52,19	13,09
1.000	Decrescentes	2.707,55	693,82	41,70	2,313
1.000	Aleatorizados	2.197,28	330,00	41,56	4,004
1.000	Aleat. c/ repetição	2.115,37	255,94	41,85	1,924
5.000	Ordenados	13,19	6,763	215,95	30,36
5.000	Decrescentes	57.661,07	7.318,96	186,64	9,084
5.000	Aleatorizados	44.905,02	4.034,53	260,77	176,99
5.000	Aleat. c/ repetição	44.792,16	4.322,43	188,73	18,30
10.000	Ordenados	21,02	1,632	609,36	189,79
10.000	Decrescentes	208.667,49	16.027,04	554,34	109,86
10.000	Aleatorizados	188.697,32	10.690,89	503,79	62,02
10.000	Aleat. c/ repetição	197.805,50	12.781,79	428,43	66,64

Fonte: elaboração própria. Médias de nove rodadas; desvio padrão = média dos desvios padrão informados em cada rodada.